SpriteFusion 2D

From generated art to game-ready pixel assets — on AMD Radeon.

Track 1: Development of Multimodal Content Creation Tools | Team: kaggle4youAMD

21 ms per sprite — 29× the host CPU

One 512×512 palette-and-grid recovery in 21 ms on an AMD Radeon (Navi 31, `gfx1100`), ROCm 7.2 / PyTorch 2.9.1 — against the same pipeline on the same host's CPU. Every figure is backed by a checked-in JSON receipt; baselines are stated in full in § 6.

1 Project Background

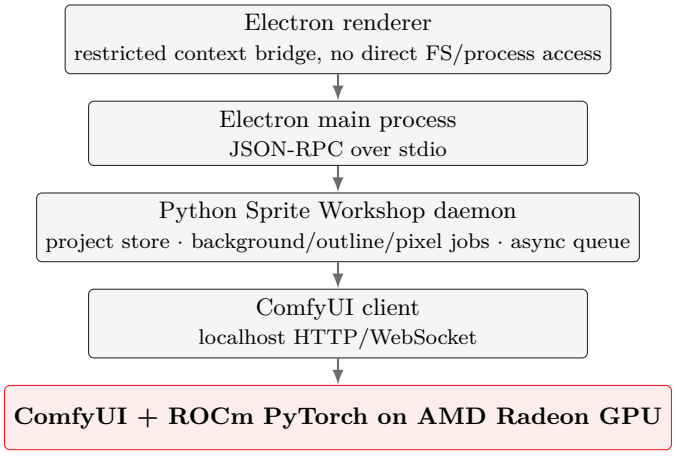
Generative image models can produce an attractive picture in seconds, but that picture is not a game asset. Studios and solo creators still need a deterministic pass that snaps the result onto a clean pixel grid, quantizes it to a small, reusable palette, cuts a clean silhouette, and produces a traceable revision history — work that is normally done by hand, image by image, in a paint program.

SpriteFusion 2D closes that gap. It is a Radeon-accelerated desktop workshop that takes generated or imported artwork and turns it into production-ready 2D game assets: transparent-background extraction, outline generation, pixel-art conversion, palette-aware grid recovery, and an integrated ComfyUI image-generation panel, all wrapped in a project/revision workflow with full undo/redo history.

2 Target Users & Application Scenarios

- **Indie and small-team game developers** who use diffusion models to rough out character, prop, and environment art and need it converted into consistent, engine-ready pixel-art sprite sheets without manual re-drawing.
- **Technical artists** who need a repeatable, auditable pipeline — every processing step is versioned as a project revision, so a palette or outline choice can be revisited or rolled back.
- **Game jam and hackathon teams** who need to go from a text prompt to a placeable sprite in minutes, on hardware they already have, without a render farm.
- **ROCm / AMD Radeon adopters** evaluating whether a full creative content pipeline — not just a single model — can live entirely on Radeon hardware.

3 System Architecture



The renderer never touches the filesystem or spawns processes directly; the Electron main process owns the Python sidecar and exposes only a narrow RPC surface. Project state and revision history are authoritative in the Python backend, and generated-image paths are served through a restricted custom protocol scoped to confirmed project directories. The same backend also ships as a standalone CLI, so the pipeline can run headless on a Radeon box with no display attached.

4 Model & Algorithm Introduction

The core of SpriteFusion 2D is a device-native PyTorch re-implementation of the pixel-snapping and palette-recovery pipeline (a from-scratch redesign built on the MIT-licensed *Sprite Fusion Pixel Snapper* concept by Hugo Duprez):

- **Grid recovery** walks the image to recover the true source pixel size, replacing a per-pixel Python loop with a vectorised, device-resident search.
- **Palette learning** runs k-means color quantization directly on GPU-resident tensors, with tiled matrix multiplication so memory stays bounded on large sheets instead of growing with image area.
- **Cell voting** assigns each recovered grid cell to its palette color as a batched tensor reduction rather than a per-cell Python comparison.
- **Outline extraction** and **background removal** share a single run-length connected-component labelling primitive so the two features never drift into separate, inconsistent implementations.

Every stage keeps the tensor on the selected device end-to-end; nothing is round-tripped to NumPy mid-pipeline. On top of this core, the ComfyUI integration submits generation jobs, polls progress, and commits a chosen candidate straight into the active project as a new revision.

5 AMD Radeon GPU / ROCm Adaptation

PyTorch exposes ROCm/HIP devices through the same `torch.cuda` API used for CUDA, so the pipeline requires no AMD-specific code path — the same `snap_images` call that runs on CPU or an NVIDIA GPU runs unmodified on Radeon. The validated Radeon target:

GPU	AMD Radeon Graphics, `gfx1100` (Navi 31)
ROCm / HIP	7.2.1 (HIP 7.2.53211)
PyTorch	2.9.1 (ROCm build), `torch.cuda.is_available() = True`
Host	Ubuntu 24.04, Python 3.12, Node.js 20

The Electron GUI itself runs on this headless Radeon host under `xvfb-run` with `-no-sandbox`, so the full desktop workflow — not just the model — has been exercised on target hardware, not only described on paper.

6 Performance on AMD Radeon

The pixel-snapping / palette-recovery core was benchmarked on the Radeon target with `benchmarks/benchmark_snapper.py`, which emits a versioned JSON *receipt* for every run: command line, environment, SHA-256 of every source file, input and output tensor, raw per-run timings, and git state. No number in this document is hand-entered — every figure below is read directly from a receipt checked into `benchmarks/receipts/`.

Configuration	CPU median (5 runs)	ROCm median (5 runs)	Speed-up
512×512 RGB, 32-color palette, pixel size 1	0.6157 s	0.0211 s	**29.2**×
512×512 RGB, 32-color palette, pixel size 6	0.6228 s	0.0219 s	**28.4**×

Protocol: one warm-up run, five synchronized measured runs, median wall time including host-to-device transfer, same input image and algorithm on both devices.

What the baseline is, and what it is not

Honesty about a speed-up ratio means stating the baseline it was divided by. The CPU column above is an AMD EPYC 9334 in a container limited to a 16-CPU quota, with PyTorch's thread pool pinned to a matching 16 threads. That pinning matters: left at its default the pool sizes itself to the physical core count, oversubscribes the quota

roughly four to one, and inflates the CPU baseline — and therefore any ratio computed against it — by about a factor of three. Each receipt records `cpu_threads` so this condition is auditable rather than assumed.

Measured against a consumer laptop CPU (Intel Core i9-13900HX) on the same input, the same ROCm result is about 8× faster, consistent with the 6.6× and 8.3× CPU-to-GPU figures this project already published in `BENCHMARKS.md`. Both baselines are reported because neither alone is the whole picture: 29× is the honest same-host comparison, 8× is what a developer on a fast desktop CPU should expect.

This measures the core tensor pipeline in isolation, not end-to-end diffusion image generation (which is dominated by the ComfyUI model's own inference cost and benefits from ROCm the same way any PyTorch model does). Correctness of the CPU/CUDA code path is covered by a golden-image regression suite (`BENCHMARKS.md`); a ROCm-specific pass of that same suite on this hardware is the next item queued behind this document.

7 Validation

- 377 Python backend tests, plus a focused 48-test slice covering the ComfyUI job lifecycle and export RPC behavior.
- 99 frontend tests in the synchronized snapshot backing this submission.
- A real, non-mocked ComfyUI submit → progress → candidate → project-commit smoke workflow, exercised on the Radeon host.
- A reproducible benchmark receipt format so every performance number in this project — including the ones above — can be independently regenerated and audited, not just quoted.

8 Roadmap

This first submission deliberately ends at image and sprite asset creation. Two extensions are already scoped but intentionally left out of the validated snapshot rather than shipped half-finished:

- **Image-to-video.** The application keeps an explicit extension point for turning a committed sprite revision into a short animation, but no video model is wired in yet — nothing here is presented as complete before it is.
- **ROCm-native golden-image pass.** The CPU/CUDA code path already passes an exact, pixel-diff-zero regression suite against the legacy reference implementation (`BENCHMARKS.md`); the same suite re-run directly on this Radeon box is queued immediately behind this document.

9 Links

- **Source & reproducibility README:** included in this pull request (`SpriteFusion2D/README.md`).
- **Benchmark receipts referenced in § 6:** `SpriteFusion2D/benchmarks/receipts/`.
- **Demo video:** `SpriteFusion2D/docs/SpriteFusion2D-demo.mp4`.

Repository code: MIT License. The pixel-snapper implementation builds on the MIT-licensed *Sprite Fusion Pixel Snapper* by Hugo Duprez, redesigned for device-native PyTorch/ROCm execution. Third-party models and ComfyUI are distributed separately under their own licenses.